

UNIVERSITÉ DE LORRAINE

Rapport de Preuve et Dédution Automatique

Implémentation de la méthode des connexions en Prolog

Étudiants :

Mathéo VIGNÈRES

Guillaume POTEL

Enseignant :

M. Didier GALMICHE

Année universitaire 2025-2026

Table des matières

Introduction	2
1 Le socle commun : Logique Propositionnelle et Premier Ordre	2
1.1 Définition des règles (Alpha et Bêta)	2
1.1.1 Le prédicat principal : <code>etiqueter/7</code>	2
1.1.2 Les règles de type Alpha (α)	2
1.1.3 Les règles de type Bêta (β)	2
1.1.4 Cas de la négation et des feuilles	3
1.2 Structures de données : Représentation des arbres	3
1.3 Prédicats utilitaires	3
2 En Logique Propositionnelle	3
2.1 Construction de l'arbre syntaxique indexé	3
2.2 Génération de l'arbre des chemins	3
2.3 Recherche de connexions et validation	3
3 En Logique du Premier Ordre (LPO)	3
3.1 Adaptation de l'arbre indexé	4
3.2 Évolution de l'arbre des chemins	4
3.3 Recherche de connexions orientée LPO	4
3.4 Unification, graphe de dépendance et multiplicité	4
Conclusion	4

Introduction

1 Le socle commun : Logique Propositionnelle et Premier Ordre

Dans cette première partie, nous présentons les éléments de base qui sont partagés entre la logique propositionnelle et la logique du premier ordre.

1.1 Définition des règles (Alpha et Bêta)

La méthode des connexions s'appuie sur l'application de règles appelée *alpha* et *beta*. Nous avons implémenté ces règles via un prédicat central : `etiqueter/7`.

Ce prédicat a pour rôle de parcourir la formule initiale pour construire un arbre indexé, où chaque nœud contient le type de règle appliquée ($\alpha, \beta, \gamma, \delta$), sa polarité, et un index unique.

1.1.1 Le prédicat principal : `etiqueter/7`

La signature du prédicat est la suivante :

```
1 etiqueter(+Formule, +Polarite, +IndexIn, -IndexOut, +Multiplicite,  
+Suffixe, -Arbre).
```

- **Polarité** : 1 pour une formule affirmée, 0 pour une formule niée.
- **IndexIn / IndexOut** : Un mécanisme de compteur permettant d'attribuer un identifiant unique (ex : $a0, a1, \dots$) à chaque nœud de l'arbre.
- **Arbre** : La structure de sortie représentant l'arbre syntaxique indexé sous forme de nœud ou de feuille.

1.1.2 Les règles de type Alpha (α)

Les règles α correspondent aux décompositions de nature conjonctive (linéaire). Dans notre implémentation, une règle α produit deux sous-arbres qui devront tous deux être validés. Par exemple, pour la conjonction de polarité 1 ($A \wedge B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 0$) :

```
1 % Exemple : Implication niee (A -> B, 0)  
2 etiqueter(A impl B, 0, IndexIn, IndexOut, M, Suffix,  
3     noeud(etiq_formule(alpha, alpha1, alpha2, A impl B, 0,  
4         NomComplet),  
5         [ArbreA, ArbreB])) :-  
6     construire_nom(IndexIn, Suffix, NomComplet),  
7     Index1 is IndexIn + 1,  
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),  
     etiqueter(B, 0, IndexMid, IndexOut, M, Suffix, ArbreB).
```

1.1.3 Les règles de type Bêta (β)

Les règles β représentent des décompositions disjonctives (branchement). Elles créent une bifurcation dans l'arbre des chemins. Nous retrouvons ici la disjonction de polarité 1 ($A \vee B, 1$) ou l'implication de polarité 0 ($A \rightarrow B, 1$) :

```

1 % Exemple : Disjonction affirmée (A ou B, 1)
2 etiqueter(A ou B, 1, IndexIn, IndexOut, M, Suffix,
3     noeud(etiq_formule(beta, beta1, beta2, A ou B, 1,
4         NomComplet),
5         [ArbreA, ArbreB])) :-
6     construire_nom(IndexIn, Suffix, NomComplet),
7     Index1 is IndexIn + 1,
8     etiqueter(A, 1, Index1, IndexMid, M, Suffix, ArbreA),
9     etiqueter(B, 1, IndexMid, IndexOut, M, Suffix, ArbreB).

```

1.1.4 Cas de la négation et des feuilles

La négation est traitée de manière transparente en inversant la polarité du reste de la formule. Ce processus se poursuit jusqu'à atteindre un **littéral** (atome). À ce stade, le prédicat crée une **feuille**, marquant la fin de la branche syntaxique.

```

1 % Traitement des atomes (Feuilles)
2 etiqueter(A, Polarite, Index, IndexOut, _, Suffix,
3     feuille(etiq_formule(atome, none, none, A, Polarite,
4         NomComplet)))) :-
5     est_litteral(A),
6     construire_nom(Index, Suffix, NomComplet),
7     IndexOut is Index + 1.

```

1.2 Structures de données : Représentation des arbres

1.3 Prédicats utilitaires

2 En Logique Propositionnelle

Cette section détaille l'implémentation de la méthode des connexions restreinte à la logique propositionnelle.

2.1 Construction de l'arbre syntaxique indexé

2.2 Génération de l'arbre des chemins

2.3 Recherche de connexions et validation

3 En Logique du Premier Ordre (LPO)

La logique du premier ordre introduit la gestion des quantificateurs, ce qui complexifie significativement la méthode des connexions.

- 3.1 Adaptation de l'arbre indexé
 - 3.2 Évolution de l'arbre des chemins
 - 3.3 Recherche de connexions orientée LPO
 - 3.4 Unification, graphe de dépendance et multiplicité
- Conclusion